On the Adaptive Security of Key-Unique Threshold Signatures

Michele Ciampi

University of Edinburgh

Elizabeth Crites

Parity Technologies

Chelsea Komlo

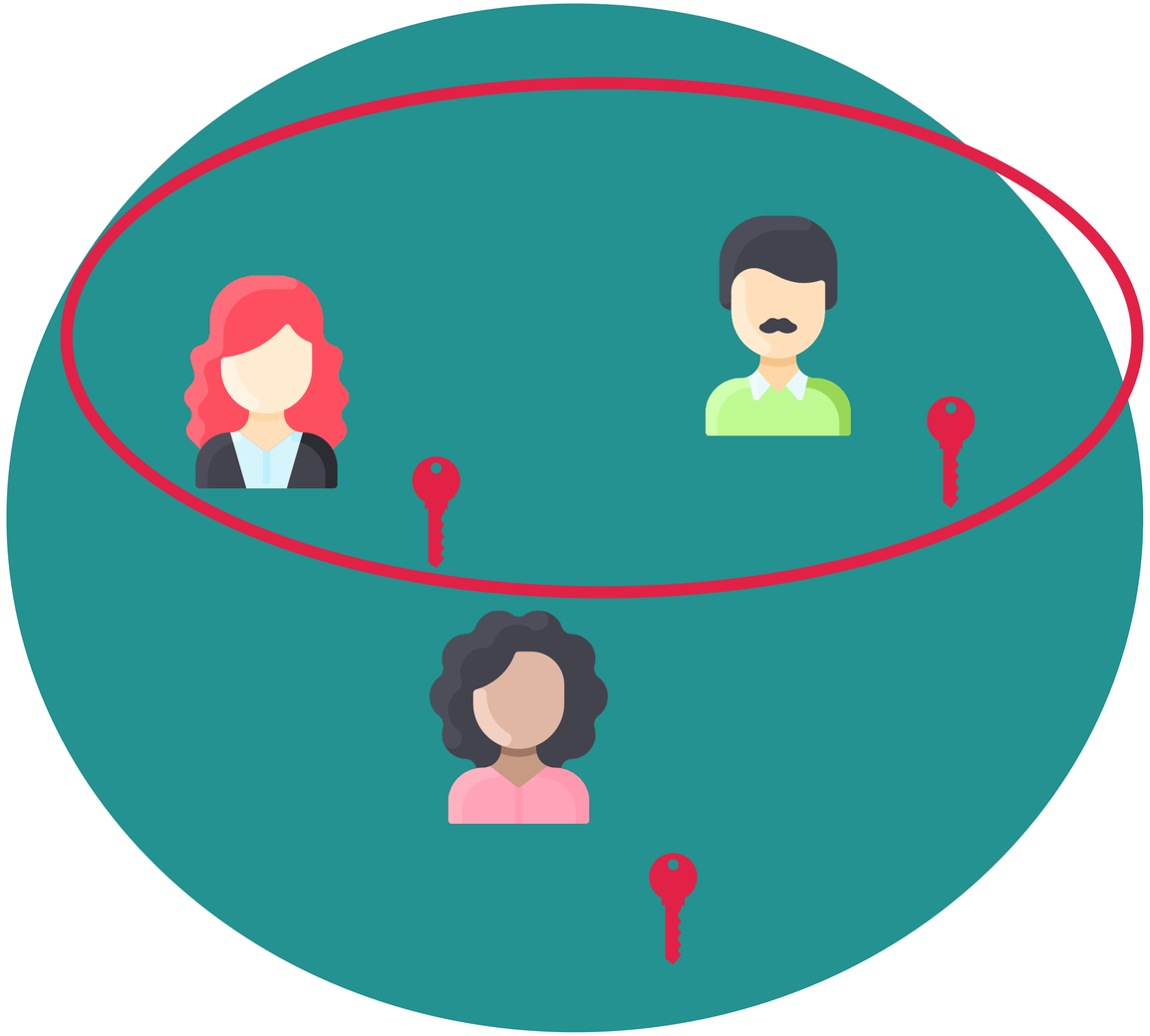
University of Waterloo,
NEAR One

Mary Maller

Inversed Tech

August 17, 2026

Threshold Signatures: Special Case of MPC



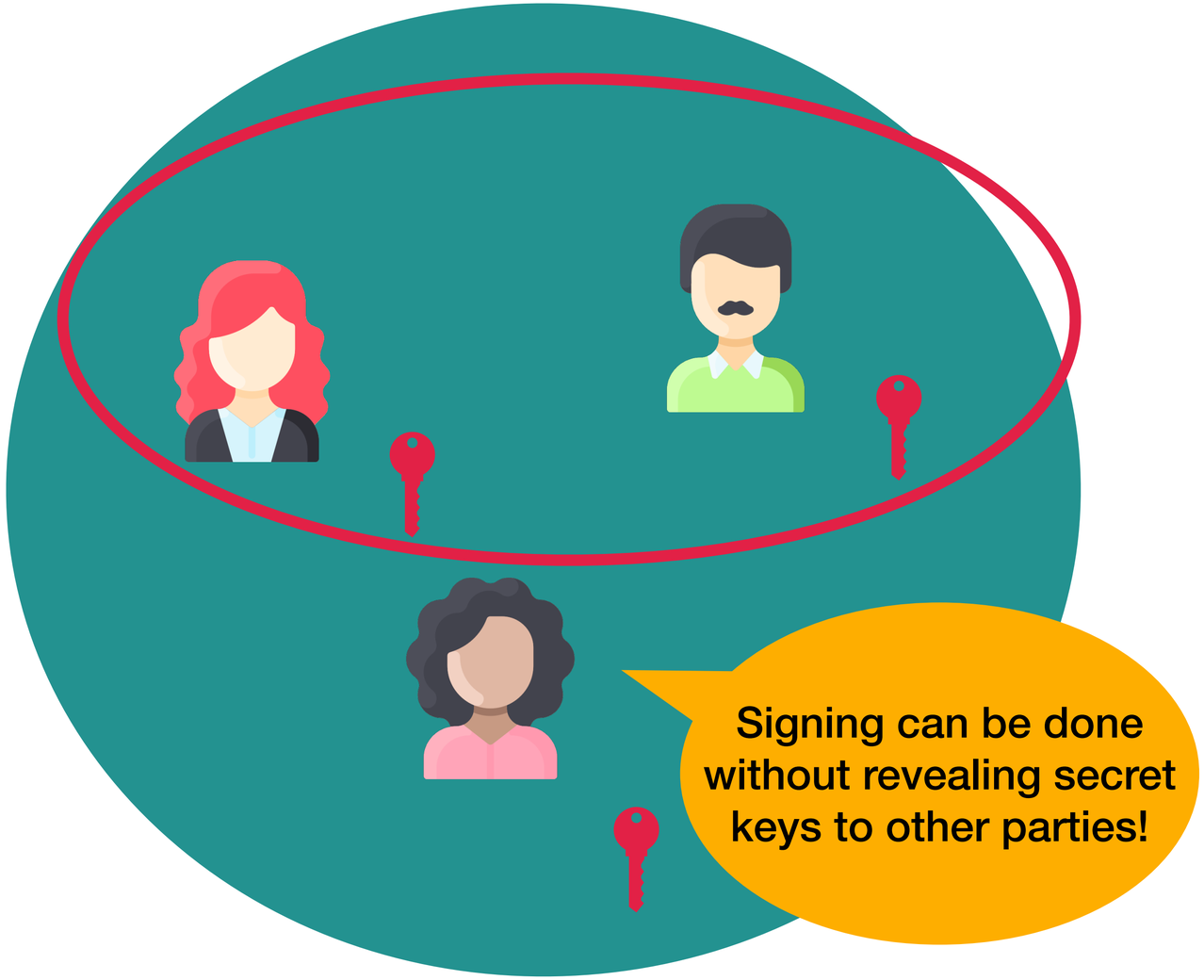


Public Key

- Ideally t-out-of-n
- Key generation via trusted dealer or DKG
- Secure up to (t-1) corruptions

(2,3) Example

Threshold Signatures: Special Case of MPC





Public Key

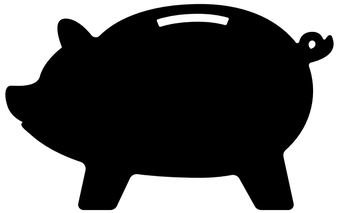
- Ideally t-out-of-n
- Key generation via trusted dealer or DKG
- Secure up to (t-1) corruptions

(2,3) Example

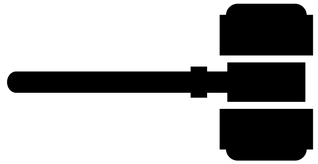
Uses for Threshold Signatures



Banks and Exchanges



Cryptocurrency Wallets



Trust Authorities (CAs)

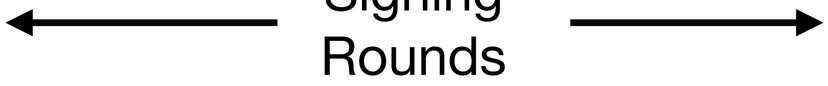
Adaptive Security

Static Corruption



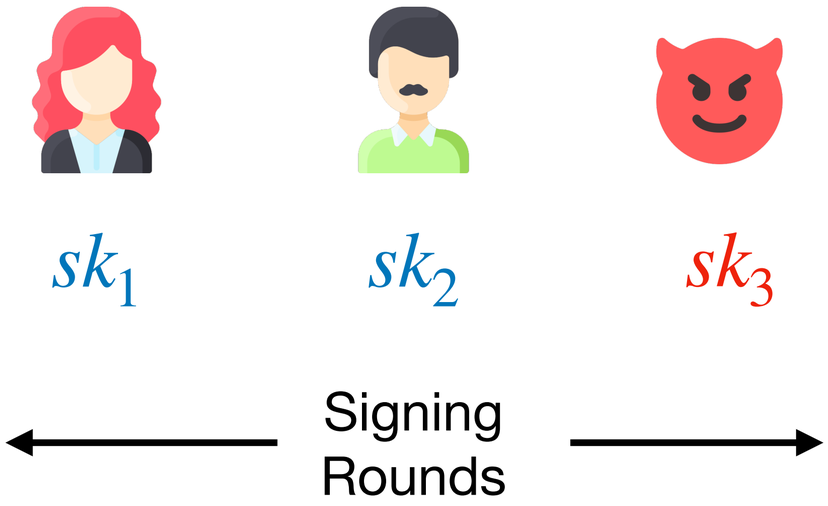


sk_1 sk_2 sk_3

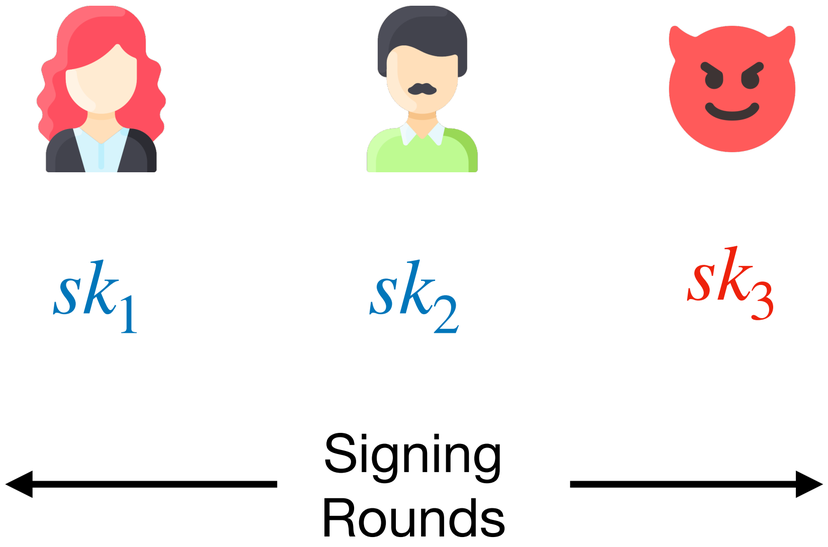


Adaptive Security

Static Corruption

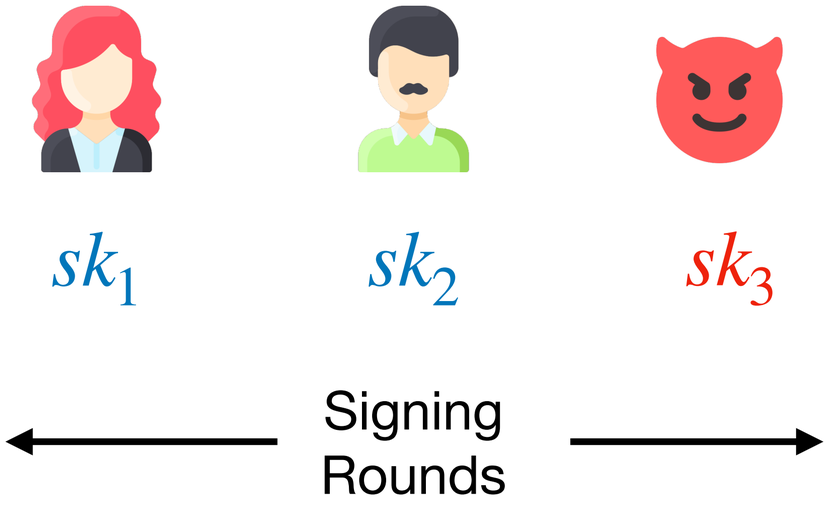


Adaptive Corruption

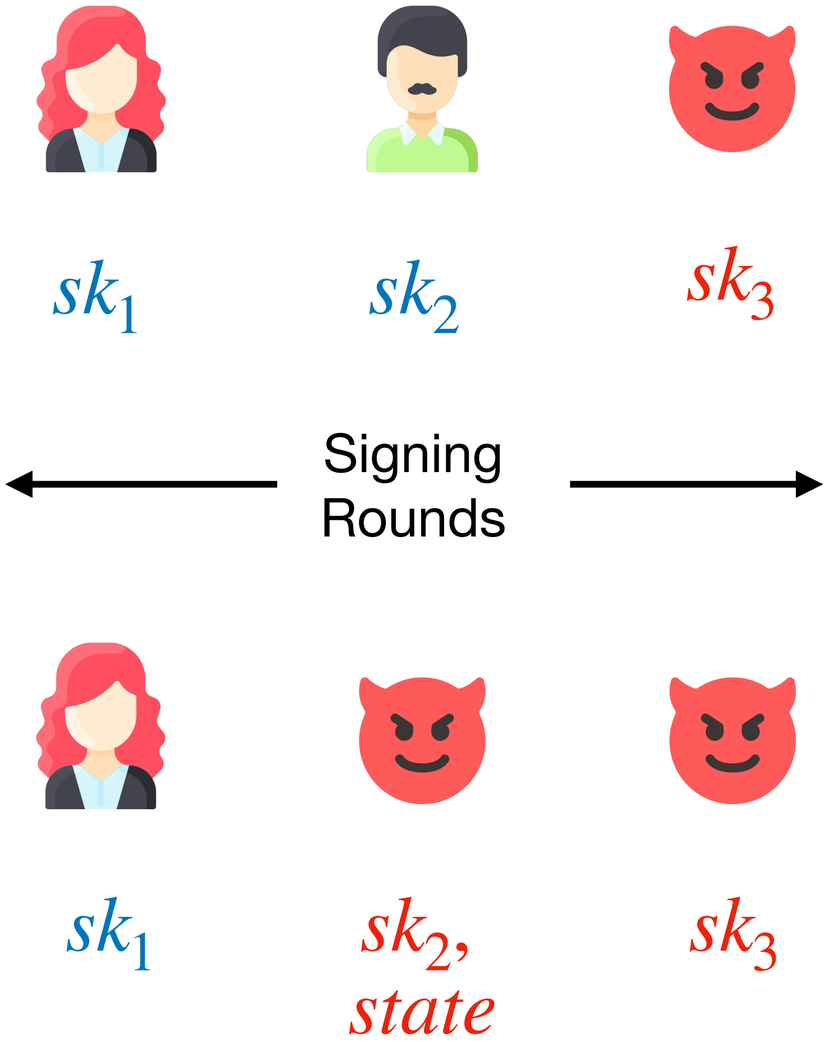


Adaptive Security

Static Corruption

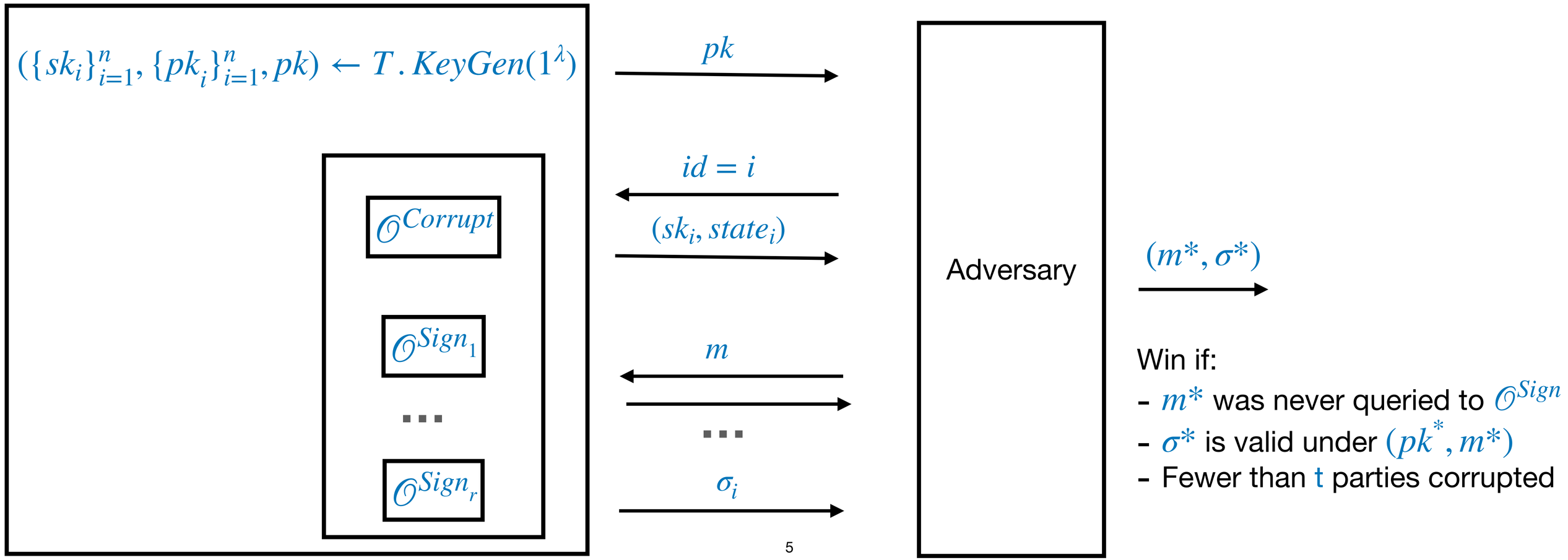


Adaptive Corruption



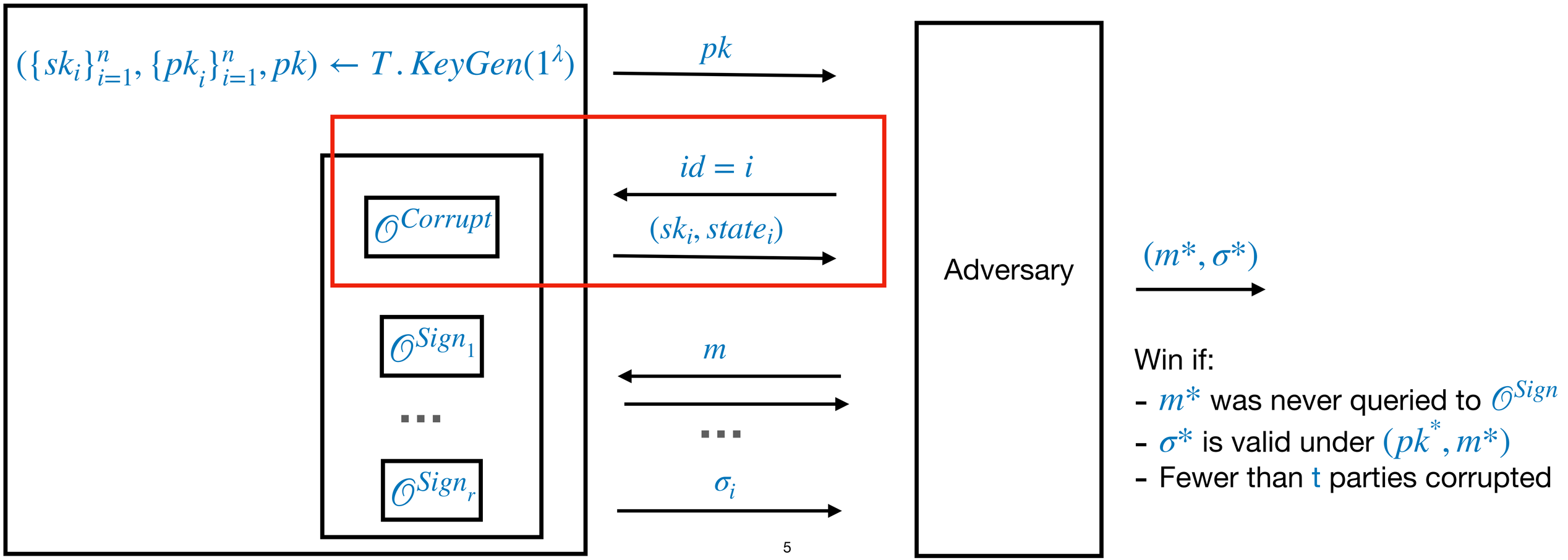
Adaptive Unforgeability

An adaptive adversary can choose on the fly which parties to corrupt. Receives not only the secret key but also the private state of the corrupted party.



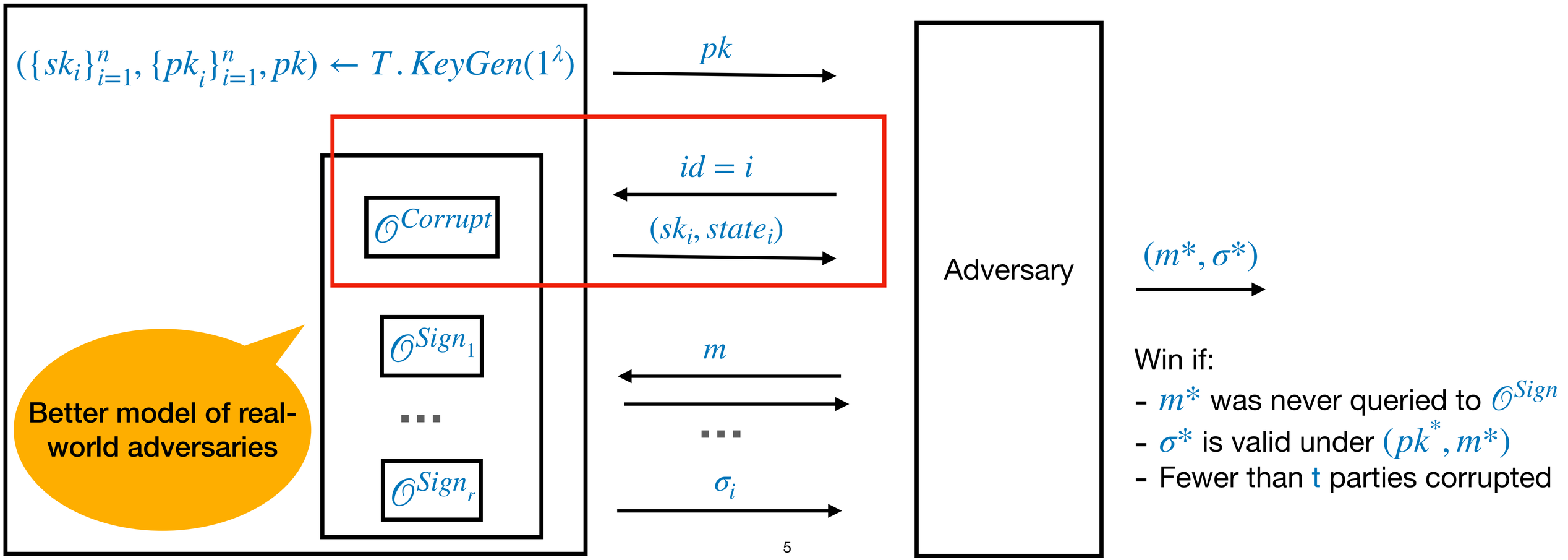
Adaptive Unforgeability

An adaptive adversary can choose on the fly which parties to corrupt. Receives not only the secret key but also the private state of the corrupted party.



Adaptive Unforgeability

An adaptive adversary can choose on the fly which parties to corrupt. Receives not only the secret key but also the private state of the corrupted party.

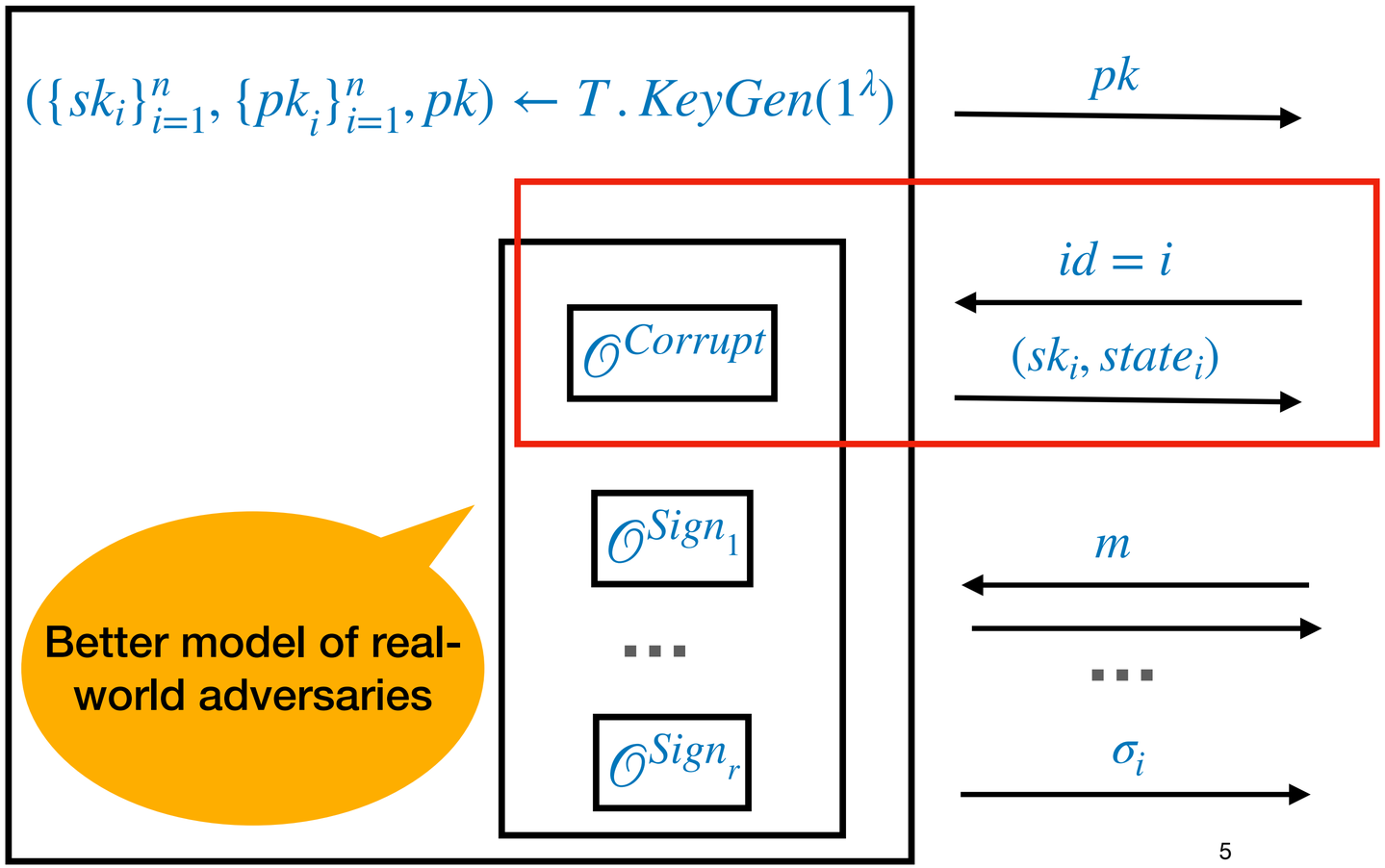


Win if:

- m^* was never queried to $\mathcal{O}^{Sign}$
- σ^* is valid under (pk^*, m^*)
- Fewer than t parties corrupted

Adaptive Unforgeability

An adaptive adversary can choose on the fly which parties to corrupt. Receives not only the secret key but also the private state of the corrupted party.



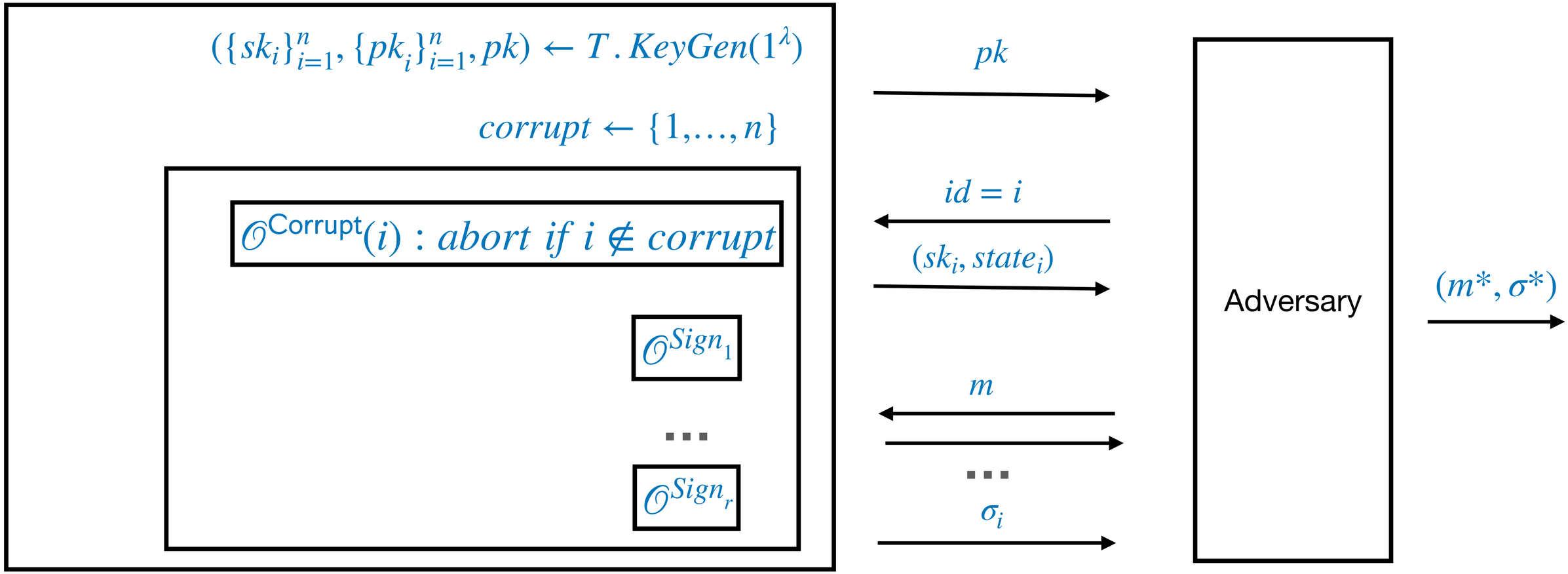


Environment must simulate:
1) Secret key shares,
2) Signing state
(over all open sessions)

(m^*, σ^*)

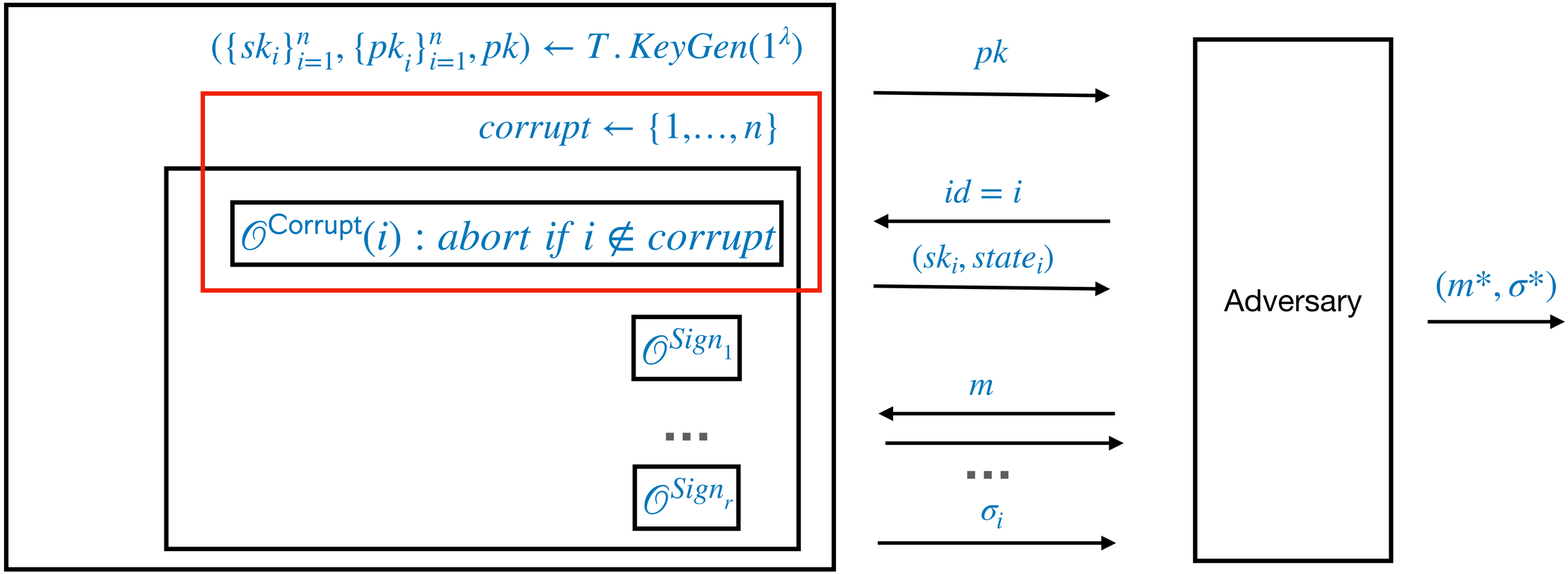
Win if:
- m^* was never queried to $\mathcal{O}^{Sign}$
- σ^* is valid under (pk^*, m^*)
- Fewer than t parties corrupted

Trivial Adaptive Unforgeability from Key-Unique Guessing



Tightness loss of $\binom{n}{t}$ in number of parties and corruption threshold.

Trivial Adaptive Unforgeability from Key-Unique Guessing



Tightness loss of $\binom{n}{t}$ in number of parties and corruption threshold.

Research Question: what security assumptions are required to prove (non-trivial) adaptive security for threshold signatures?

Our Results

Definition

Key-Unique Threshold Signatures

Impossibility Result #1

Impossible under Non-Interactive Computational or Decisional Assumptions

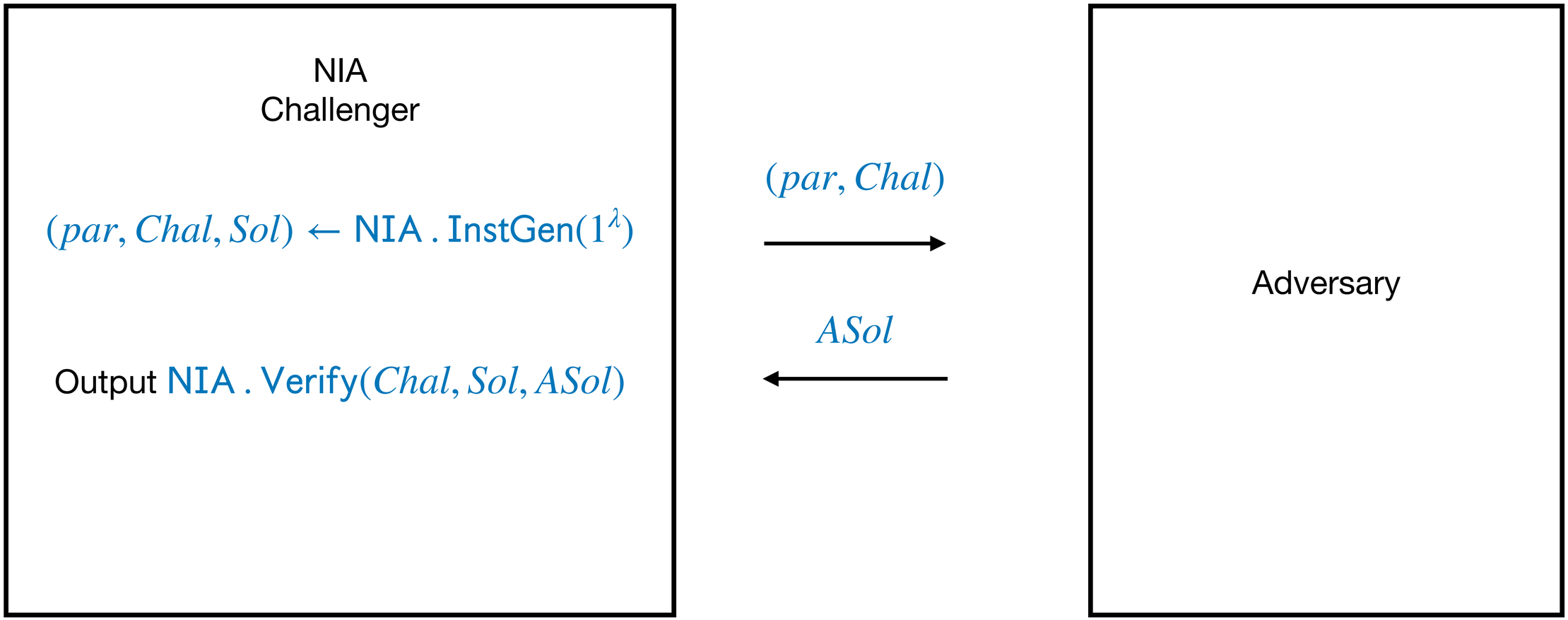
Key-Unique Schnorr Threshold Signatures

Impossibility Result #2

Impossible with rewinding under Interactive (One-More) DL

Background

Non-Interactive Computational or Decisional Assumption (NIA)



Non-Interactive Computational or Decisional Assumption (NIA)

NIA
Challenger

$(par, Chal, Sol) \leftarrow \mathsf{NIA.InstGen}(1^{\lambda})$

Output $\mathsf{NIA.Verify}(Chal, Sol, ASol)$

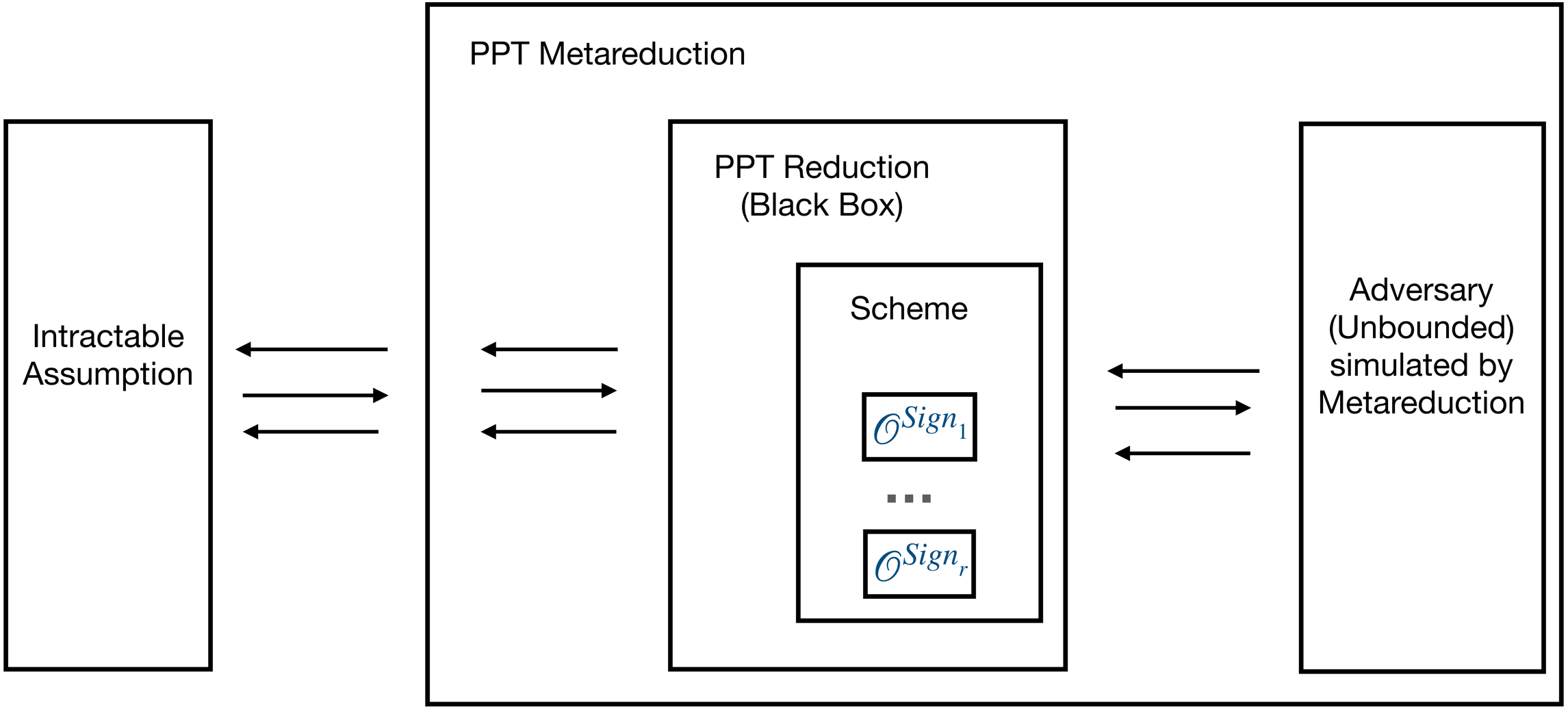
$(par, Chal)$

$ASol$

Examples

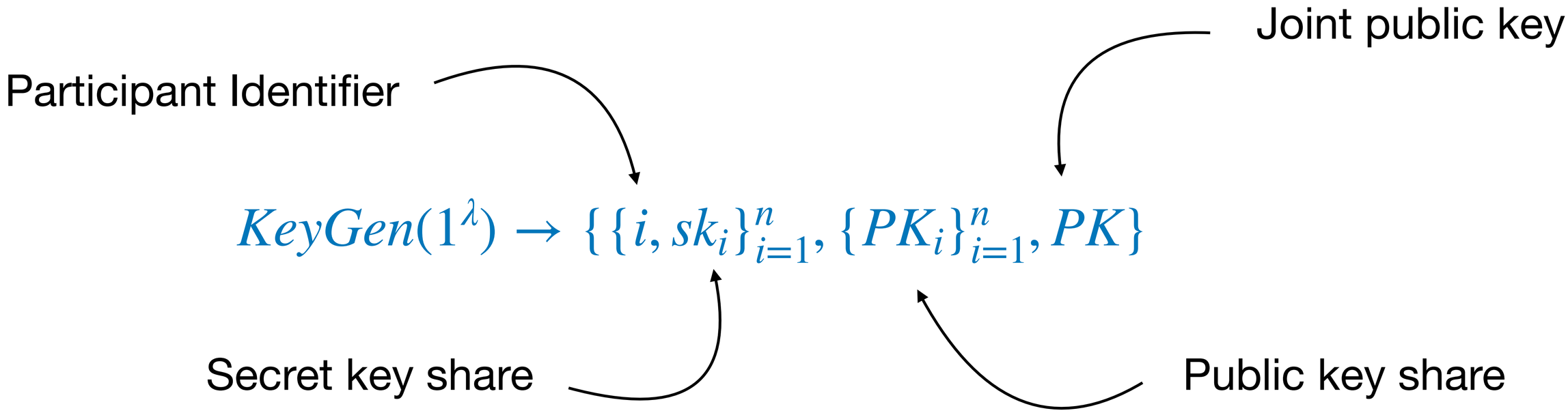
- Discrete Logarithm Assumption (DL)
- Computational Diffie Hellman Assumption (CDH)
- Decisional Diffie-Hellman Assumption (DDH)
- q-Strong Diffie-Hellman Assumption (q-SDH)
- q-Bilinear Diffie-Hellman Inversion Assumption (q-BDHI)

Metareduction: Prove impossibility of a reduction



Key-Uniqueness

Key Generation for Threshold Signatures



Key Uniqueness

Property 1: Unique correspondence

Property 2: Efficient verification

Key Uniqueness

Property 1: Unique correspondence

Public key and shares have a unique correspondence via an (efficient) injective map f.

Group public key

Label

$$f(0, sk) \rightarrow PK$$

Public key share

$$f(i, sk_i) \rightarrow PK_i$$

Key Uniqueness

Property 2: Efficient verification

An efficient Verify algorithm exists to check that public keys are well-formed.

If

$$VerifyPKs(PK, \{PK_i\}_{i=1}^{n}\} \rightarrow 1$$

Then

$$Recover(\{sk_j\}_{j \in S}\}) \rightarrow sk$$

Key Uniqueness

Property 2: Efficient verification

An efficient Verify algorithm exists to check that public keys are well-formed.

If

$$VerifyPKs(PK, \{PK_i\}_{i=1}^{n}\} \rightarrow 1$$

Then

$$Recover(\{sk_j\}_{j \in S}\}) \rightarrow sk$$

Our simulation fails against a reduction that outputs invalid keys

Example #1: Perfectly Binding Commitment





Key unique. Employed by: FROST/2/3 [**K**G20, BC**K**MTZ22, C**K**M21, CGRS23], Sparkle [C**K**M23], Lindell22, ZeroS [M23]

Example #1: Perfectly Binding Commitment





Step 1: Sample $(sk, \{a_1, \ldots, a_{t-1}\}) \leftarrow \mathbb{Z}_q^t$

Key unique. Employed by: FROST/2/3 [**K**G20, BC**K**MTZ22, C**K**M21, CGRS23], Sparkle [C**K**M23], Lindell22, ZeroS [M23]

Example #1: Perfectly Binding Commitment





Step 1: Sample $(sk, \{a_1, \ldots, a_{t-1}\}) \leftarrow \mathbb{Z}_q^t$

Step 2: Define $p(x) = sk + \sum_{j=1}^{t-1} a_j x^j$

Key unique. Employed by: FROST/2/3 [KG20, BCKMTZ22, CKM21, CGRS23], Sparkle [CKM23], Lindell22, ZeroS [M23]

Example #1: Perfectly Binding Commitment





Step 1: Sample $(sk, \{a_1, \ldots, a_{t-1}\}) \leftarrow \mathbb{Z}_q^t$

Step 2: Define $p(x) = sk + \sum_{j=1}^{t-1} a_j x^j$

Step 3: Define $sk_i = p(i)$

Key unique. Employed by: FROST/2/3 [**K**G20, BC**K**MTZ22, C**K**M21, CGRS23], Sparkle [C**K**M23], Lindell22, ZeroS [M23]

Example #1: Perfectly Binding Commitment





Step 1: Sample $(sk, \{a_1, \ldots, a_{t-1}\}) \leftarrow \mathbb{Z}_q^t$

Step 2: Define $p(x) = sk + \sum_{j=1}^{t-1} a_j x^j$

Step 3: Define $sk_i = p(i)$

Step 4: Define $PK_i = g^{p(i)} = g^{sk_i}$ and $PK = g^{p(0)} = g^{sk}$

Key unique. Employed by: FROST/2/3 [**K**G20, BC**K**MTZ22, C**K**M21, CGRS23], Sparkle [C**K**M23], Lindell22, ZeroS [M23]

Example #1: Perfectly Binding Commitment





Step 1: Sample $(sk, \{a_1, \ldots, a_{t-1}\}) \leftarrow \mathbb{Z}_q^t$

Shamir secret sharing

Step 2: Define $p(x) = sk + \sum_{j=1}^{t-1} a_j x^j$

Step 3: Define $sk_i = p(i)$

Step 4: Define $PK_i = g^{p(i)} = g^{sk_i}$ and $PK = g^{p(0)} = g^{sk}$

Key unique. Employed by: FROST/2/3 [**K**G20, BC**K**MTZ22, C**K**M21, CGRS23], Sparkle [C**K**M23], Lindell22, ZeroS [M23]

Example #1: Perfectly Binding Commitment





Step 1: Sample $(sk, \{a_1, \ldots, a_{t-1}\}) \leftarrow \mathbb{Z}_q^t$

Shamir secret sharing

Step 2: Define $p(x) = sk + \sum_{j=1}^{t-1} a_j x^j$

Step 3: Define $sk_i = p(i)$

Step 4: Define $PK_i = g^{p(i)} = g^{sk_i}$ and $PK = g^{p(0)} = g^{sk}$

Key unique. Employed by: FROST/2/3 [**K**G20, BC**K**MTZ22, C**K**M21, CGRS23], Sparkle [C**K**M23], Lindell22, ZeroS [M23]

$((i, sk_i), \{PK_i\}_{i=1}^n, PK)$

Example #1: Perfectly Binding Commitment





Step 1: Sample $(sk, \{a_1, \ldots, a_{t-1}\}) \leftarrow \mathbb{Z}_q^t$

Shamir secret sharing

Step 2: Define $p(x) = sk + \sum_{j=1}^{t-1} a_j x^j$

Step 3: Define $sk_i = p(i)$

Step 4: Define $PK_i = g^{p(i)} = g^{sk_i}$ and $PK = g^{p(0)} = g^{sk}$

Key-uniqueness allows for identifiable abort “for free.”

…3 [**K**G20, BC**K**MTZ22, C**K**M21, …, ZeroS [M23]

$((i, sk_i), \{PK_i\}_{i=1}^n, PK)$

Example #2: No Public Key Shares





Step 1: Sample $(sk, \{a_1, \ldots, a_{t-1}\}) \leftarrow \mathbb{Z}_q^t$

Step 2: Define $E(x) = sk + \sum_{j=1}^{t-1} a_j x^j$

Step 3: Define $sk_i = E(i)$

Example #2: No Public Key Shares





Step 1: Sample $(sk, \{a_1, \ldots, a_{t-1}\}) \leftarrow \mathbb{Z}_q^t$

Step 2: Define $E(x) = sk + \sum_{j=1}^{t-1} a_j x^j$

Step 3: Define $sk_i = E(i)$

Step 4: Define $PK = g^{E(0)} = g^{sk}$

Example #2: No Public Key Shares





Step 1: Sample $(sk, \{a_1, \ldots, a_{t-1}\}) \leftarrow \mathbb{Z}_q^t$

Step 2: Define $E(x) = sk + \sum_{j=1}^{t-1} a_j x^j$

Step 3: Define $sk_i = E(i)$

Step 4: Define $PK = g^{E(0)} = g^{sk}$

$((i, sk_i), PK)$ →

Example #2: No Public Key Shares





Step 1: Sample $(sk, \{a_1, \ldots, a_{t-1}\}) \leftarrow \mathbb{Z}_q^t$

Step 2: Define $E(x) = sk + \sum_{j=1}^{t-1} a_j x^j$

Step 3: Define $sk_i = E(i)$

Step 4: Define $PK = g^{E(0)} = g^{sk}$

Not key unique. Employed by: KRT24

$f(0, sk) \rightarrow g^{sk} = PK$

$f(i, sk_i) \rightarrow \perp$

$((i, sk_i), PK)$

Example #2: No Public Key Shares





Step 1: Sample $(sk, \{a_1, \ldots, a_{t-1}\}) \leftarrow \mathbb{Z}_q^t$

Step 2: Define $E(x) = sk + \sum_{j=1}^{t-1} a_j x^j$

Step 3: Define $sk_i = E(i)$

Step 4: Define $PK = g^{E(0)} = g^{sk}$

Tradeoff: Extra machinery for identifiable abort

Not key unique. Employed by: KRT24

$f(0, sk) \rightarrow g^{sk} = PK$

$f(i, sk_i) \rightarrow \perp$

$((i, sk_i), PK)$

Example #3: Pedersen Commitments





Example #3: Pedersen Commitments





Step 1: Sample $sk \leftarrow \mathbb{Z}_q, \{a_{1,i} \ldots a_{1,t-1}\} \leftarrow \mathbb{Z}_q^{t-1}, \{a_{2,i} \ldots a_{2,t-1}\} \leftarrow \mathbb{Z}_q^{t-1}$

Example #3: Pedersen Commitments





Step 1: Sample $sk \leftarrow \mathbb{Z}_q, \{a_{1,i} \ldots a_{1,t-1}\} \leftarrow \mathbb{Z}_q^{t-1}, \{a_{2,i} \ldots a_{2,t-1}\} \leftarrow \mathbb{Z}_q^{t-1}$

Step 2: Define $E_i(x) = sk + \sum_{j=1}^{t-1} a_{1j} x^j$ and $F(x) = 0 + \sum_{j=1}^{t-1} a_{2j} x^j$

Example #3: Pedersen Commitments





Step 1: Sample $sk \leftarrow \mathbb{Z}_q, \{a_{1,i} \ldots a_{1,t-1}\} \leftarrow \mathbb{Z}_q^{t-1}, \{a_{2,i} \ldots a_{2,t-1}\} \leftarrow \mathbb{Z}_q^{t-1}$

Step 2: Define $E_i(x) = sk + \sum_{j=1}^{t-1} a_{1j} x^j$ and $F(x) = 0 + \sum_{j=1}^{t-1} a_{2j} x^j$

Step 3: Define $\bar{sk}_i = (E(i), F(i))$

Example #3: Pedersen Commitments





Step 1: Sample $sk \leftarrow \mathbb{Z}_q, \{a_{1,i} \ldots a_{1,t-1}\} \leftarrow \mathbb{Z}_q^{t-1}, \{a_{2,i} \ldots a_{2,t-1}\} \leftarrow \mathbb{Z}_q^{t-1}$

Step 2: Define $E_i(x) = sk + \sum_{j=1}^{t-1} a_{1j} x^j$ and $F(x) = 0 + \sum_{j=1}^{t-1} a_{2j} x^j$

Step 3: Define $\bar{sk}_i = (E(i), F(i))$

Step 4: Define $PK_i = g^{E(i)} h^{F(i)}$ and $PK = g^{E(0)} = g^{sk}$

Example #3: Pedersen Commitments





Step 1: Sample $sk \leftarrow \mathbb{Z}_q, \{a_{1,i} \ldots a_{1,t-1}\} \leftarrow \mathbb{Z}_q^{t-1}, \{a_{2,i} \ldots a_{2,t-1}\} \leftarrow \mathbb{Z}_q^{t-1}$

Step 2: Define $E_i(x) = sk + \sum_{j=1}^{t-1} a_{1j} x^j$ and $F(x) = 0 + \sum_{j=1}^{t-1} a_{2j} x^j$

Step 3: Define $\bar{sk}_i = (E(i), F(i))$

Step 4: Define $PK_i = g^{E(i)} h^{F(i)}$ and $PK = g^{E(0)} = g^{sk}$

$((i, \bar{sk}_i), \{PK_i\}_{i=1}^n, PK)$ →

Example #3: Pedersen Commitments





Step 1: Sample $sk \leftarrow \mathbb{Z}_q, \{a_{1,i} \ldots a_{1,t-1}\} \leftarrow \mathbb{Z}_q^{t-1}, \{a_{2,i} \ldots a_{2,t-1}\} \leftarrow \mathbb{Z}_q^{t-1}$

Step 2: Define $E_i(x) = sk + \sum_{j=1}^{t-1} a_{1j} x^j$ and $F(x) = 0 + \sum_{j=1}^{t-1} a_{2j} x^j$

Step 3: Define $\bar{sk}_i = (E(i), F(i))$

Step 4: Define $PK_i = g^{E(i)} h^{F(i)}$ and $PK = g^{E(0)} = g^{sk}$

$((i, \bar{sk}_i), \{PK_i\}_{i=1}^n, PK)$ →

Not key unique. Employed by: DR24

Example #3: Pedersen Commitments





Step 1: Sample $sk \leftarrow \mathbb{Z}_q, \{a_{1,i} \ldots a_{1,t-1}\} \leftarrow \mathbb{Z}_q^{t-1}, \{a_{2,i} \ldots a_{2,t-1}\} \leftarrow \mathbb{Z}_q^{t-1}$

Step 2: Define $E_i(x) = sk + \sum_{j=1}^{t-1} a_{1j} x^j$ and $F(x) = 0 + \sum_{j=1}^{t-1} a_{2j} x^j$

Step 3: Define $\bar{sk}_i = (E(i), F(i))$

Step 4: Define $PK_i = g^{E(i)} h^{F(i)}$ and $PK = g^{E(0)} = g^{sk}$

Tradeoff:
Identifiable abort via additional NIZK.

$((i, \bar{sk}_i), \{PK_i\}_{i=1}^n, PK)$

Not key unique. Employed by: DR24

Impossibility Results

Main Result #1

key-unique + NIA = adaptive security above $t/2$
not possible!

(for a broad class of security reductions)

Non-Interactive Computational or Decisional Assumption

NIAs

~all ROM

Scheme	Static Security	Adaptive Security	Impossibility Results Apply
Threshold BLS (I) [19]	GDH	✗	1
Threshold BLS (I) [7]	-	OMDL+AGM (full)	1
Threshold BLS (III) [43]	co-CDH	co-CDH+DDH (full)	✗*
Libert et al. [67]	-	SXDH†	✗*
Threshold Waters [78]	-	CDH† (full)	✗*
Threshold RSA [2]	RSA†	RSA† (full)	✗*
HBTS-Mask [33]	-	DL (full)	✗*
Threshold ECDSA [55]	Various†	✗	1
Threshold ECDSA [29]	Various	✗	1
Threshold BBS+ [45]	Various†	✗	1
Twinkle$_1$ [9]	-	AOMCDH (full)	1
Twinkle$_2$ [9]	-	DDH (full)	✗*
Dazzle/-T [32]	-	DDH (full)	✗*
FROST/2/3 [65, 41, 14, 75, 35, 38]	(A)OMDL	AOMDL+LDVR+AGM (full) AOMDL ($t/2$)	1,2
FROST-Mask [33]	-	AOMDL (full)	✗*
ms-FROST [4]	-	AOMDL+AGM (full)	✗*
FaFROST [11]	-	-	✗*
Sparkle+ [40]	DL	AOMDL ($t/2$)	1,2
[34]	CDL	✗	1,2
Lindell [68]	Schnorr, $\mathcal{F}_{\mathsf{zk}}$	✗	1,2
Classic S. [70]	DL	✗	1,2
Zero S. [70]	DL	DL (full, erasures)	✗*
Crackle & Snap [62]	-	DL (full)	✗*
Glacius [6]	-	DDH (full)	✗*
Gargos [5]	-	DDH (full)	✗*
GCRS [57]	-	DDH (full)	✗*
Abe-Fehr [1]	-	Schnorr+DDH (full)	✗*
Stinson-Strobl [77]	Schnorr	✗	1,2
ROAST [75]	OMDL	✗	1,2
SPRINT [17]	DL	✗	1,2
HARTS [8]	-	-	1,2
GKMN [54]	n-party Schnorr, $\mathcal{F}_{\mathsf{zk}}^{R_{\mathrm{DL}}}$	✗	1,2
Arctic [64]	DL	✗	1,2

Schnorr schemes

✗* = not key unique

impossible in ROM and AGM

Main Result #2

interactive

key-unique Schnorr + (A)OMDL/OMCDH = adaptive security above $t/2$ with rewinding **not possible!**

(A)OMDL

~all ROM

Scheme	Static Security	Adaptive Security	Impossibility Results Apply
Threshold BLS (I) [19]	GDH	✗	1
Threshold BLS (I) [7]	-	OMDL+AGM (full)	1
Threshold BLS (III) [43]	co-CDH	co-CDH+DDH (full)	✗*
Libert et al. [67]	-	SXDH†	✗*
Threshold Waters [78]	-	CDH† (full)	✗*
Threshold RSA [2]	RSA†	RSA† (full)	✗*
HBTS-Mask [33]	-	DL (full)	✗*
Threshold ECDSA [55]	Various†	✗	1
Threshold ECDSA [29]	Various	✗	1
Threshold BBS+ [45]	Various†	✗	1
Twinkle$_1$ [9]	-	AOMCDH (full)	1
Twinkle$_2$ [9]	-	DDH (full)	✗*
Dazzle/-T [32]	-	DDH (full)	✗*
FROST/2/3 [65, 41, 14, 75, 35, 38]	(A)OMDL	AOMDL+LDVR+AGM (full) AOMDL ($t/2$)	1,2
FROST-Mask [33]	-	AOMDL (full)	✗*
ms-FROST [4]	-	AOMDL+AGM (full)	✗*
FaFROST [11]	-	-	✗*
Sparkle+ [40]	DL	AOMDL ($t/2$)	1,2
[34]	CDL	✗	1,2
Lindell [68]	Schnorr, $\mathcal{F}_{\mathsf{zk}}$	✗	1,2
Classic S. [70]	DL	✗	1,2
Zero S. [70]	DL	DL (full, erasures)	✗*
Crackle & Snap [62]	-	DL (full)	✗*
Glacius [6]	-	DDH (full)	✗*
Gargos [5]	-	DDH (full)	✗*
GCRS [57]	-	DDH (full)	✗*
Abe-Fehr [1]	-	Schnorr+DDH (full)	✗*
Stinson-Strobl [77]	Schnorr	✗	1,2
ROAST [75]	OMDL	✗	1,2
SPRINT [17]	DL	✗	1,2
HARTS [8]	-	-	1,2
GKMN [54]	n-party Schnorr, $\mathcal{F}_{\mathsf{com-zk}}^{R_{\mathrm{DL}}}$	✗	1,2
Arctic [64]	DL	✗	1,2

Schnorr schemes

(A)OMDL common

rewinding common

Takeaways

Takeaways

- Adaptive security is a stronger notion of unforgeability for threshold signatures; closer to a real-world adversary.

Takeaways

- Adaptive security is a stronger notion of unforgeability for threshold signatures; closer to a real-world adversary.
- However, proving adaptive security under standard(-ish) assumptions is challenging.

Takeaways

- Adaptive security is a stronger notion of unforgeability for threshold signatures; closer to a real-world adversary.
- However, proving adaptive security under standard(-ish) assumptions is challenging.
- Our results frame how adaptively secure schemes can be constructed, and classify the tradeoffs incurred (key-uniqueness).

Takeaways

- Adaptive security is a stronger notion of unforgeability for threshold signatures; closer to a real-world adversary.
- However, proving adaptive security under standard(-ish) assumptions is challenging.
- Our results frame how adaptively secure schemes can be constructed, and classify the tradeoffs incurred (key-uniqueness).

Thank you!